DAY 3

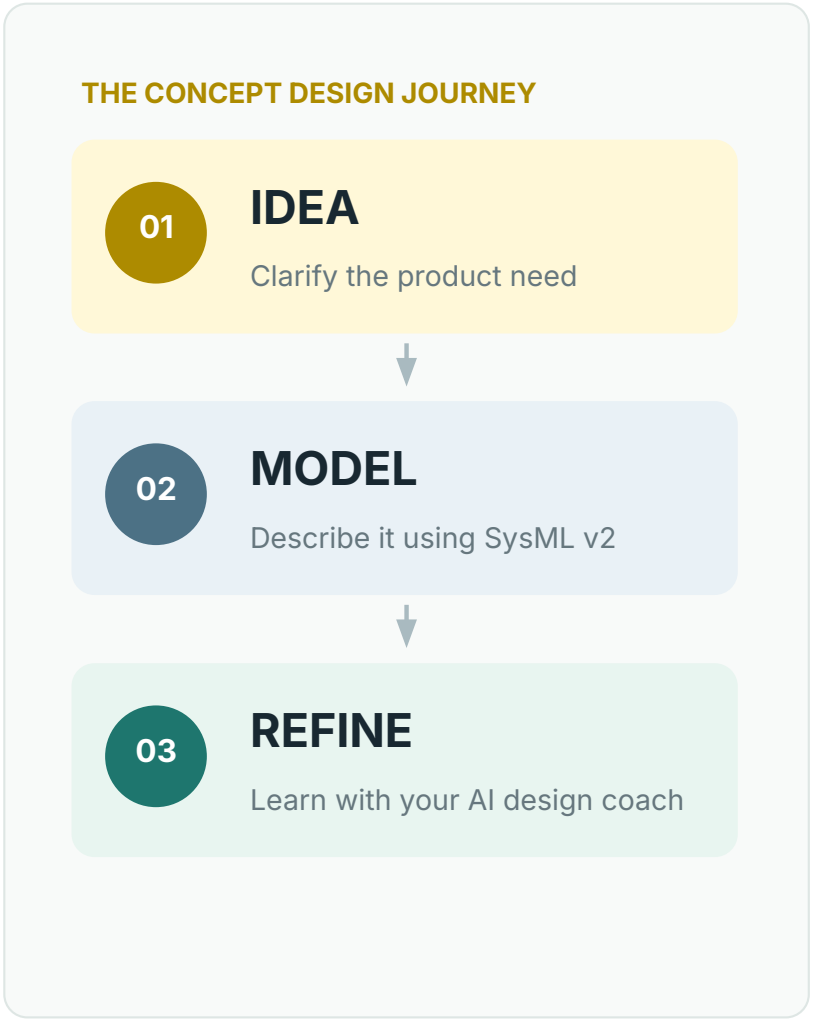
From Ideas to Concepts

SysML v2 for Product Concepts

Abstract design / SysML v2 / AI design coach

Laurent Balmelli · Hidekazu Nishimura

Keio University | October 2026



DAY 3 / AGENDA

2 PERIODS

Day 3 schedule

From an abstract product concept to your own SysML v2 model, guided by AI.

Product concepts and primitives

09:00–10:00 — 60 MIN

- Visual Concept Making methodology and the Idea Wall
- SysML v2 textual notation: requirements, structure, behavior
- Personalized AI tutor, configured agents, and design skills

Class discussion and student presentations

10:00–10:30 — 30 MIN

- Introduce your product idea selected on Day 2 (2–3 min each)

Break

10:30–10:45 — 15 MIN

Practice: build your product concept

10:45–12:15 — 90 MIN

- Create a technology-independent concept with requirements, structure, and behavior
- Use Jupyter, OpenCode, your AI tutor, and the concept-design skill

SECTION 3.1.1 / VISUAL CONCEPT MAKING

3 MIN

Visual Concept Making: design the concept first

Begin with the problem and feasibility assumptions, not a shopping list of parts.

01 / FRAME

Start from the problem

Who needs the product? What meaningful outcome should it enable?

Idea Wall: make collaboration easier

02 / QUESTION

Make feasibility explicit

Which capabilities must be possible for the idea to work?

Capture ideas anywhere; find related ideas

03 / ABSTRACT

Keep options open

Describe functions, behavior and minimal properties before selecting technology.

Capture / Store / Display / Analyze

KEY PRINCIPLE **Define WHAT the product must achieve before choosing HOW to build it.**

SECTION 3.1.2 / FROM IDEA TO CONCEPT

3 MIN

From product idea to SysML v2 concept

Use the Idea Wall to see how an idea becomes a technology-independent model.

TODAY'S OBJECTIVE **How to transform a product idea into a technology-independent SysML v2 concept, with an AI agent acting as a design coach.**

PRODUCT IDEA / IDEA WALL

Share and explore ideas

Feasibility assumptions

- Capture and share contributions from anywhere
- Recognize related ideas and patterns

→

ABSTRACT CAPABILITIES / LOGICAL ELEMENTS

Capture
Receive contributions

Store
Maintain idea information

Display
Present shared ideas

Analyze
Detect patterns and relationships

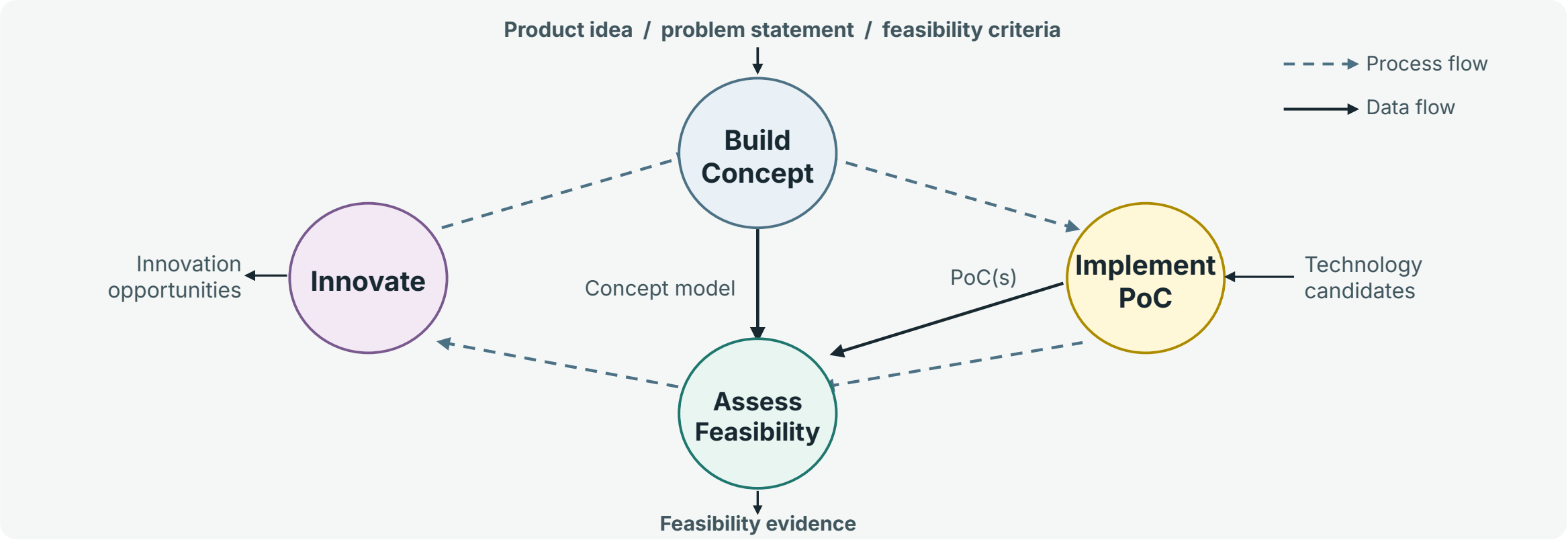
DAY 3 Describe WHAT it must do **DAY 4 Explore HOW it could be realized**

SECTION 3.1.3 / THE DESIGN LOOP

4 MIN

Visual Concept Making is an iterative loop

The concept, realizations and feasibility criteria are refined together.



SECTION 3.2.1 / MODELING PERSPECTIVES

2 MIN

From product concepts to SysML v2

One product concept can be described through several connected modeling perspectives.

1

01 / REQUIREMENTS

What must it do?

Capture ideas from anywhere

2

02 / STRUCTURE

What is it made of?

Capture / Store / Display / Analyze

3

03 / BEHAVIOR

How does it work?

Capture → Store → Display

4

04 / SPECIFICATIONS

How well must it work?

Capacity and response criteria

ONE IDEA WALL CONCEPT MODEL

Different elements, consistent meaning and traceability

SECTION 3.2.2 / REQUIREMENTS

3 MIN

Modeling requirements: the Idea Wall

Convert product assumptions into explicit requirements and feasibility criteria.

IDEA WALL / REQUIREMENT DEFINITIONS

```
package IdeaWallRequirements {
  private import ScalarValues::*;
  requirement def <'R1'> SharedIdeaCapture {
    doc /* Contributors shall capture ideas from
           anywhere and share them on
           collaborative team boards. */
  }
  requirement def <'FC1'> MultipleFormCapture {
    doc /* The system shall support capturing
           ideas in at least two formats,
           including text. */
    attribute supportedFormatCount : Integer;
    require constraint { supportedFormatCount >= 2 }
  }
  requirement sharedCapture : SharedIdeaCapture;
  requirement multipleFormCapture : MultipleFormCapture;
}
```

FROM ASSUMPTION TO REQUIREMENT

R1 SharedIdeaCapture

A clear statement of intended capability

FC1 MultipleFormCapture

A minimum number of supported formats

FORMAL: at least two formats
PROSE: one must be text

SECTION 3.2.3 / REQUIREMENTS PRIMITIVES

2 MIN

SysML v2 primitives: requirements

Every highlighted expression in the preceding example has a specific modeling role.

PRIMITIVE	DEFINITION	IDEA WALL EXAMPLE
package	Organizes model elements in a namespace	package IdeaWallRequirements
import	Makes external definitions available	private import ScalarValues::*
requirement def	Defines a reusable requirement	requirement def SharedIdeaCapture
<'R1'>	Short identifier for model traceability	<'R1'>
doc	Human-readable intent attached to a model element	doc /* ... */
attribute	Typed property or requirement parameter	supportedFormatCount : Integer
require constraint	Condition the requirement must satisfy	supportedFormatCount >= 2
requirement usage	Uses a previously defined requirement	sharedCapture : SharedIdeaCapture

TEACHING QUESTION / Which part is formal and checkable, and which part expresses intent in prose?

SECTION 3.2.4 / LOGICAL STRUCTURE

4 MIN

Modeling logical structure: the Idea Wall

Definitions and usages describe abstract elements without choosing technologies.

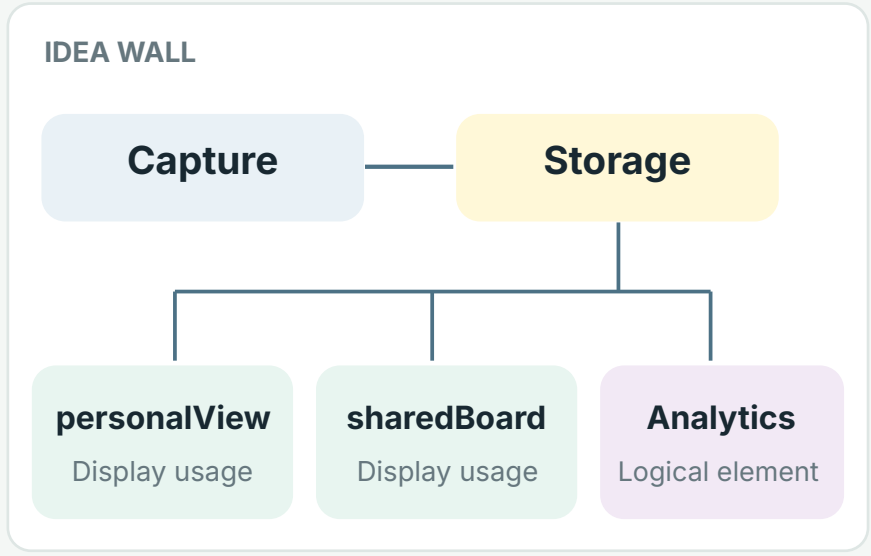
IDEA WALL / LOGICAL STRUCTURE

```
package IdeaWallStructure {
  part def Capture;
  part def Storage;
  part def Display;
  part def Analytics;

  part def IdeaWall {
    part capture : Capture;
    part storage : Storage;
    part personalView : Display;
    part sharedBoard : Display;
    part analytics : Analytics;

    connect capture to storage;
    connect storage to personalView;
    connect storage to sharedBoard;
    connect storage to analytics;
  }
  part ideaWall : IdeaWall;
}
```

CONCEPTUAL ASSEMBLY



Connections are undirected; data flow is modeled separately.

SECTION 3.2.5 / STRUCTURE PRIMITIVES

2 MIN

SysML v2 primitives: logical structure

Definitions describe kinds of elements; usages describe their roles inside a product.

PRIMITIVE	MEANING	IDEA WALL EXCERPT
package	Groups related elements	package IdeaWallStructure
part def	Defines a reusable logical or physical element	part def Display;
part	Declares an element usage in context	part sharedBoard : Display;
:	Specifies the type of a usage	sharedBoard : Display
nested part	Builds the product composition	part storage : Storage;
connect … to …	Establishes a structural connection	connect capture to storage;

DEFINITION part def Display;

USAGE part sharedBoard : Display;

SECTION 3.2.6 / BEHAVIOR

4 MIN

Modeling behavior: capture an idea

Actions express what happens and which activities must follow others.

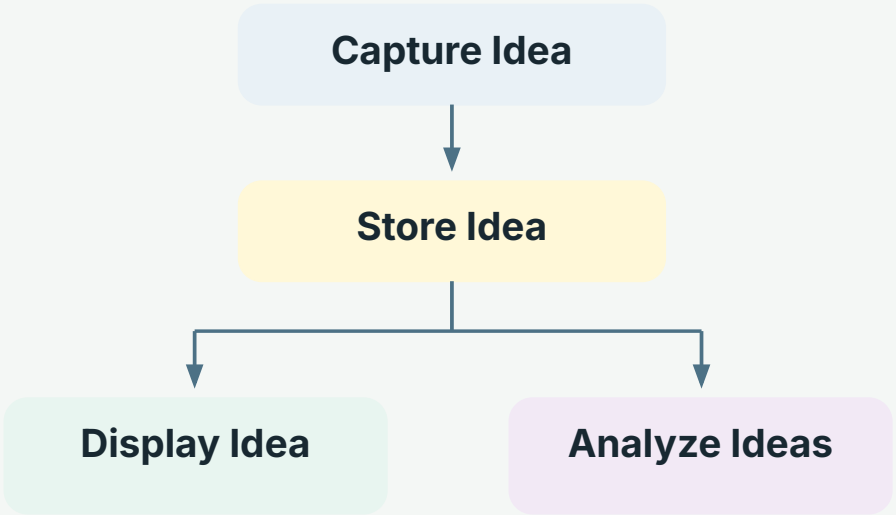
IDEA WALL / ACTION MODEL

```
package IdeaWallBehavior {
  action def CaptureIdea;
  action def StoreIdea;
  action def DisplayIdea;
  action def AnalyzeIdeas;

  action def ProcessIdea {
    action capture : CaptureIdea;
    action store : StoreIdea;
    action display : DisplayIdea;
    action analyze : AnalyzeIdeas;

    first capture then store;
    first store then display;
    first store then analyze;
  }
  action processIdea : ProcessIdea;
}
```

SIMPLIFIED BEHAVIOR



Both follow Store; their relative order is unspecified.

SECTION 3.2.7 / BEHAVIOR PRIMITIVES

2 MIN

SysML v2 primitives: behavior

Actions are reusable behaviors; successions establish their required ordering.

PRIMITIVE	MEANING	IDEA WALL EXCERPT
action def	Defines a reusable behavior	action def CaptureIdea;
action	Declares a usage of a behavior	action capture : CaptureIdea;
:	Types the usage by its definition	capture : CaptureIdea
nested action	Decomposes a larger process	action store : StoreIdea;
first ... then	Specifies an ordering constraint	first capture then store;
top-level usage	Introduces an action in the package	action processIdea : ProcessIdea;

REMEMBER **Successions establish order; they do not by themselves transfer data or guarantee parallelism.**

SECTION 3.2.8 / MINIMAL SPECIFICATION

4 MIN

From a logical element to a minimal specification

Describe required capabilities and limits, while leaving the implementation open.

BEFORE / LOGICAL ELEMENT

```
part def Display;
```

What kind of element is this?

The definition says little about its capabilities.

AFTER / MINIMALLY SPECIFIED DISPLAY

```
part def Display {
  doc /* Present ideas and refresh the view. */
  attribute visibleIdeaCapacity : Integer;
  attribute updateLatencySeconds : Real;
  perform action displayIdea :
    IdeaWallBehavior::DisplayIdea;
  assert constraint {
    visibleIdeaCapacity >= 10 and
    updateLatencySeconds <= 2.0
  }
}
```

KEY IDEA **Minimal does not mean vague: specify WHAT must be achieved, not HOW to implement it.**

SECTION 3.2.9 / SPECIFICATION PRIMITIVES

2 MIN

SysML v2 primitives: specifications

A minimal specification combines existing constructs rather than introducing a new one.

PRIMITIVE	MEANING	IDEA WALL EXCERPT
doc	States the intended responsibility	doc /* Present ideas ... */
attribute	Declares a property that can be constrained	visibleIdeaCapacity : Integer
Integer / Real	Scalar value types from the library	Integer; Real
perform action	Links a part to behavior it can perform	displayIdea : DisplayIdea
assert constraint	States a condition the element must obey	visibleIdeaCapacity >= 10
>= / <= / and	Comparison and logical operators	latency <= 2.0
::	Qualifies a name across namespaces	IdeaWallBehavior::DisplayIdea

DAY 3 OUTPUT → an abstract concept with enough detail to compare realizations on Day 4

SECTION 3.3.1 / YOUR AI TUTOR

2 MIN

Meet your personal AI design coach

A specialized tutor helps you learn SysML v2 as you build your own product models.



ONE STUDENT. ONE PERSONAL TUTOR.

Ask, explain, experiment, and improve while modeling.

SYSML v2

Explain

Understand concepts and textual notation.

Ask

Check your reasoning with short questions.

Guide

Explore alternatives and receive hints.

Verify

Review models and check their syntax.

YOUR ROLE The AI helps you learn. You make the design decisions.

SECTION 3.3.2 / LEARNING THROUGH DIALOGUE

2 MIN

Learning through dialogue

Sometimes your tutor asks why before showing you how.

IDEA WALL / A SHORT EXCHANGE

STUDENT I want to add a Display element to my Idea Wall.

AI TUTOR Would you use part def or part for a reusable Display type? Why?

STUDENT part def, because several displays can share the same definition.

AI TUTOR Exactly. What must Display do, independent of the technology?

WHY DIALOGUE HELPS

01 **Explain your choice**

Show how you reason.

02 **Get a useful hint**

Learn without losing ownership.

03 **See your progress**

Review questions and revisions.

Private learning history supports reflection.

REMEMBER **Your reasoning matters as much as the model you create.**

SECTION 3.3.3 / SPECIALIZED AGENT

1 MIN

How OpenCode becomes a SysML specialist

Course configuration, reliable references, and modeling tools work together.



Jupyter Notebook + OpenCode

STUDENT INTERFACE

One classroom workspace to model, visualize, and work with your tutor.

01 Agent configuration

Tutoring role, interaction style, and classroom instructions.

02 Reference knowledge

SysML v2 specification, examples, and course material.

03 Modeling tools

Inspect, edit, visualize, and validate SysML v2 models.

RESULT **A general-purpose agent becomes your specialized SysML tutor.**

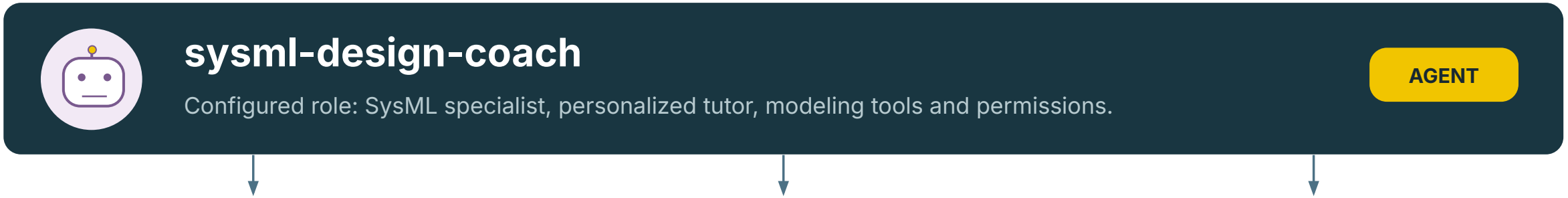
Classroom interaction recording is a separate part of the learning environment.

SECTION 3.4.1 / CONFIGURED AGENTS AND SKILLS

2 MIN

Configured agents vs. agent skills

The agent has a role and tools. A skill provides a reusable procedure.



AGENT	Who helps you and what it can do
SKILL	How a particular task is performed

SECTION 3.4.2 / SKILL LIFECYCLE

3 MIN

How an agent skill works

An ordinary SKILL.md file turns a repeatable method into instructions for OpenCode.

EXAMPLE / CONCEPT DESIGN SKILL

.opencode/skills/concept-design/SKILL.md

```
---
name: concept-design
description: Guide SysML v2 concept design
---
# Visual Concept Making
1. Clarify product idea and users
2. Identify feasibility criteria
3. Define abstract functions
4. Model structure and behavior
5. Define minimal specifications
Ask students to justify choices.
```

01 **DISCOVER**

The agent sees the skill name and description.

02 **LOAD**

OpenCode reads the detailed SKILL.md when selected.

03 **APPLY**

The tutor guides the student using that methodology.

KEY IDEA **A skill provides a method, not a new language model.**

SECTION 3.4.3 / SKILL 1 - DAY 3

4 MIN

Skill 1: concept-design

Build the abstract product first. Keep implementation decisions for later.

01	**Define the problem**	Clarify why the product is needed.
02	**Set feasibility criteria**	What must the concept demonstrate?
03	**Identify abstract functions**	Describe capabilities, not devices.
04	**Model the concept**	Use SysML v2 structure and behavior.
05	**Set minimal specifications**	Define what each element must provide.

IDEA WALL / ABSTRACT CONCEPT

- **Capture**
- **Storage**
- **Display**
- **Analytics**

A COACHING QUESTION

What must Display do, without choosing a screen or app?

OUTPUT **A technology-independent SysML v2 concept.**

SECTION 3.4.4 / SKILL 2 - DAY 4

3 MIN

Skill 2: poc-realization

Discover concrete technologies that could satisfy the conceptual specifications.

01	**Read the specifications**	Extract required properties and behavior.
02	**Find candidate technologies**	Search hardware, software and services.
03	**Compare alternatives**	Check fit, interfaces and limitations.
04	**Assemble PoC options**	Propose multiple concrete realizations.
05	**Document the choices**	Track candidates back to the concept.

IDEA WALL / DISPLAY CANDIDATES

1. **Large screen**
2. **Tablet display**
3. **Web interface**

Research candidates; verify capabilities and compatibility.

OUTPUT **Sourced PoC options with documented technology choices.**

SECTION 3.4.5 / SKILL 3 - DAY 4

3 MIN

Skill 3: feasibility-assessment

Compare each PoC with the concept and look for evidence, gaps, and innovation.

01	**Retrieve the concept**	Review the original feasibility criteria.
02	**Inspect each PoC**	Identify parts, interfaces and test results.
03	**Assess coverage**	Separate verified results from claims.
04	**Identify the gaps**	Locate missing functions or constraints.
05	**Explore innovation**	Improve, invent, or refine the concept.

IDEA WALL / ILLUSTRATIVE REVIEW

CONCEPT CRITERION	STATUS
Multiple capture formats	PARTIAL
Display 10+ ideas	PROPOSED
Update within 2 sec	UNTESTED
Group similar ideas	GAP

A candidate is not yet a validated solution.

KEY INSIGHT **Innovation begins where existing solutions fall short.**

SECTION 3.5 / CLASS DISCUSSION

30 MIN

Your product idea: tell us about it

Explain the problem before proposing the technology that might solve it.

TODAY / SHORT INTRODUCTIONS

2–3 minutes per student · Idea selected on Day 2

01 THE PROBLEM

The problem

Who experiences the problem?
Why does it matter?

WHO / WHY

02 THE IDEA

The idea

What should your product accomplish?

WHAT

03 THE CHALLENGE

The challenge

Which feasibility question is most important?

WHAT NEEDS TESTING?

DESIGN PRINCIPLE **Focus on capabilities, not implementation choices.**

SECTION 3.6 / GUIDED MODELING PRACTICE

90 MIN

Practice: build your product concept

Use your SysML design coach and concept-design to create a technology-independent model.

01 — 10 MIN

Get started

Open Jupyter, select the SysML coach, and load concept-design.

02 — 15 MIN

Define the problem

State the idea, feasibility assumptions, and first criteria.

03 — 20 MIN

Build logical structure

Define abstract parts, usages, and logical connections.

04 — 20 MIN

Model behavior

Describe an important user scenario or action sequence.

05 — 15 MIN

Specify capabilities

Add properties, behaviors, and minimal constraints.

06 — 10 MIN

Review and refine

Validate the model, inspect views, and list open questions.

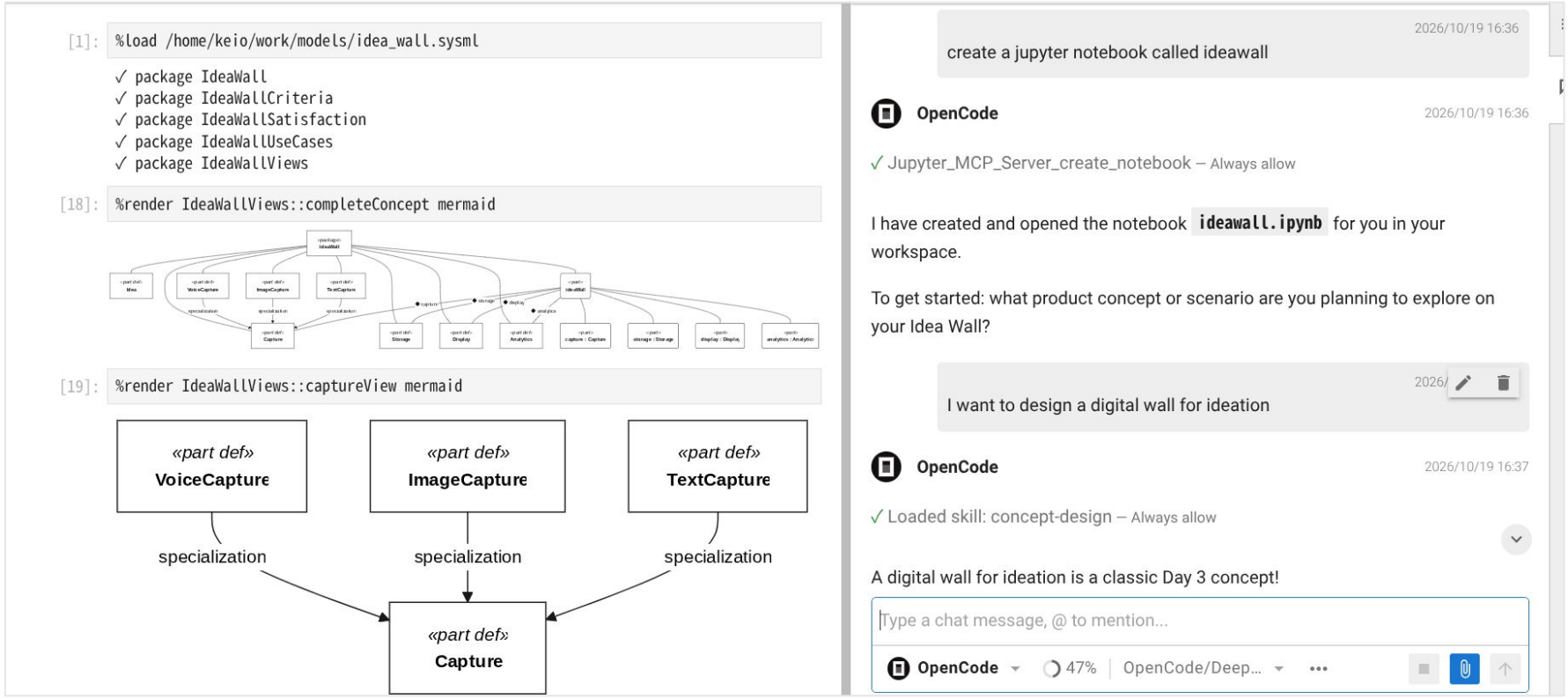
YOUR OUTPUT **Requirements & criteria · Structure · Behavior · Minimal specifications**

SECTION 3.7 / GUIDED MODELING PRACTICE

3 MIN

How to start your concept model

Start with your agent, create a notebook for display, and open your model for review



慶應義塾 Keio University

DAY 3 / END

Model the concept. Question the choices.

A good concept keeps possibilities open.

Define / Model / Explain / Refine

Questions?

NEXT / DAY 4: PRODUCT REALIZATION

THREE KEY TAKEAWAYS

01 **WHY CONCEPTS?**
Describe what, not how

02 **WHY SYSML V2?**
Make the design explicit

03 **WHY AN AI TUTOR?**
Learn by explaining choices